Proposal: Construction.AI — Knowledge-Graph + Agentic System for Material Takeoff, Cut Optimization, and Build Instructions

Jason Cusati
Construction.AI Project

***Abstract*—This proposal describes a system that reads architectural plans and produces (1) an accurate bill of materials (BOM), (2) optimized cut lists that reduce waste (e.g., selecting 12ft stock for an 11ft 6in cut instead of 16ft), and (3) build instructions aligned with construction practices and applicable codes. The system uses a Knowledge Graph (KG) to store structured information about components, materials, fasteners, building rules, jurisdiction-specific regulations, and historical project outcomes. A set of LLM-driven agents then queries the KG to infer assemblies, validate compliance, and generate outputs in formats consumable by lumber yard and procurement systems.**

I. Motivation and Problem Statement

Material takeoff—the process of extracting quantities from architectural plans—remains a critical bottleneck in construction estimation. Estimators must infer assembly intent (e.g., header configurations, king stud packs, blocking patterns) from plan conventions and building codes. Even with accurate BOMs, material waste remains high without stock-aware cut optimization. Downstream stakeholders (framers, suppliers, project managers) require machine-readable outputs and detailed build instructions, not just raw quantities.

A. Industry Challenges

The construction industry faces persistent productivity gaps [1]:

- **Time burden**: Professional estimators spend 50–80% of their time on quantity takeoffs, limiting the number of bids they can process [2]. A typical 2,000 sq.ft. residential frame takeoff requires 8–12 hours of manual work.
- **Error rates**: Manual measuring and counting leads to 10–15% error rates in material estimates [3], resulting in costly change orders, project delays, and strained supplier relationships. Common errors include miscounting studs for wall intersections, incorrect header sizing, and missing blocking requirements.
- **Inconsistency**: Two estimators working from the same plans often produce BOMs differing by 5–10% in key materials like lumber and fasteners, undermining cost predictability.
- **Knowledge gaps**: Junior estimators lack deep knowledge of framing methodology and jurisdiction-specific code requirements (IRC [4], local amendments), leading to non-compliant or over-conservative designs.
- **Material waste**: Without cut optimization, lumber waste rates of 15–20% are typical; optimized systems demonstrate waste reduction to under 5% [5], representing significant cost savings on large projects.

B. Goals

Goal: Create an end-to-end workflow that:

(a) Interprets plans into *component assemblies* (walls, headers, openings)
(b) Selects *real-world purchasable stock* (lengths, grades, treatments)
(c) Computes *cuts with waste minimization* using optimization algorithms
(d) Exports data in *supplier-friendly formats* (CSV, JSON, EDI)
(e) Generates *build instructions* with code compliance citations

II. Related Work and Contributions

This section surveys the relevant literature across five domains—knowledge graphs, LLM-based agents, cut-stock optimization, building code automation, and construction document parsing—before presenting our key contributions.

A. Knowledge Graphs in Construction

Knowledge graphs (KGs) have emerged as a powerful paradigm for representing complex domain knowledge with explicit relationships [6]. In the Architecture, Engineering, and Construction (AEC) sector, Zheng et al. [7] provide a comprehensive review of KG applications, identifying use cases in project management, safety monitoring, and facility operations.

Several works have explored linking BIM data with semantic technologies. Pauwels and Terkaj [8] developed ifcOWL to convert IFC building models to OWL ontologies, enabling SPARQL-based queries. Wu et al. [9] demonstrated natural-language retrieval over BIM object databases using KG embeddings. For construction safety, Fang et al. [10] combined computer vision with ontologies to identify site hazards.

More recently, Pan et al. [11] presented a roadmap for integrating KGs with large language models, identifying three paradigms: KG-enhanced LLMs, LLM-augmented KGs, and synergized approaches. Edge et al. [12] introduced

GraphRAG, demonstrating that graph-structured retrieval outperforms naive RAG for complex reasoning tasks.

Gap: While KGs have been applied to BIM retrieval and safety monitoring, no prior work uses a KG as the central backbone for material takeoff, cut optimization, and compliance checking in an integrated system.

B. LLM-Based Autonomous Agents

The emergence of capable LLMs has enabled a new paradigm of autonomous agents that reason, plan, and use tools [13], [14]. Li et al. [15] survey agentic LLM systems, categorizing architectures by memory, planning, and tool-use capabilities.

A key enabling pattern is ReAct [16], which interleaves reasoning traces with action execution, improving both performance and interpretability. Schick et al. [17] demonstrated that LLMs can learn to invoke external tools (calculators, search engines, APIs) to augment their capabilities. Zhang et al. [18] provide a comprehensive survey of tool-learning approaches, including structured function calling now standard in Claude [19] and GPT-4 [20].

Orchestration frameworks like LangChain [21] provide abstractions for building multi-step agent workflows with memory and tool integration. Zhu et al. [22] specifically examine LLMs for KG construction and reasoning, showing promise for automated knowledge extraction.

Gap: While multi-agent systems have been applied to general reasoning tasks, no prior work deploys specialized agent architectures for construction domain tasks like component inference, code compliance, and procurement optimization.

C. Cut-Stock Optimization

The cutting stock problem (CSP), introduced by Gilmore and Gomory [23], seeks to minimize waste when cutting stock material into required pieces. Wäscher et al. [24] provide a comprehensive typology of cutting and packing problems, distinguishing one-dimensional (lumber), two-dimensional (panels), and three-dimensional variants.

For lumber optimization specifically, Cui et al. [25] developed heuristics for multiple stock lengths, a common scenario in construction where 8ft, 10ft, 12ft, and 16ft boards are standard. Modern solvers like Google OR-Tools [26] provide integer linear programming (ILP) capabilities suitable for exact solutions on moderately-sized problems.

In construction automation, Manrique et al. [5] developed FRAMEX/CUTEX for CAD-based wood framing and cut optimization, demonstrating waste reduction of 15–20% compared to manual methods.

Gap: Existing optimizers operate in isolation from the takeoff process. Decisions about which stock to cut are disconnected from the plan facts and code rules that generated the requirements, preventing root-cause analysis of waste.

D. Building Code Compliance Automation

Automated compliance checking has been a long-standing goal in construction informatics. Solihin and Eastman [27] classified compliance rules by complexity, from simple attribute checks to complex spatial reasoning. Dimyadi and Amor [28] surveyed the state of automated code checking, identifying challenges in rule formalization and model quality.

NLP-based approaches have shown promise for extracting requirements from regulatory text. Zhang and El-Gohary [29], [30] developed semantic NLP pipelines to extract compliance rules from building codes like the IRC [4]. Beach et al. [31] demonstrated rule-based semantic approaches for automated regulatory compliance in construction.

Gap: Prior compliance systems either require manual rule encoding or focus on BIM model checking. None integrate compliance verification with material takeoff and provide citations traceable to specific code sections.

E. Construction Document Parsing

Automatic analysis of architectural floor plans has received significant attention. Pizarro et al. [32] survey methods ranging from traditional image processing to deep learning approaches. Benchmark datasets like CubiCasa5K [33] have enabled systematic evaluation of room segmentation and wall detection.

For symbol detection, Rezvanifar et al. [34] applied YOLO-based object detection to identify doors, windows, and fixtures in floor plans. Jamieson et al. [35] extended this work with construction-specific symbol vocabularies. Tools like PyMuPDF [36] enable extraction of vector graphics from PDF plans, while EasyOCR [37] handles dimension text recognition.

Gap: Existing parsers focus on room layout and basic elements. None extract the full information needed for material takeoff: wall types, opening schedules, structural annotations, and dimension chains.

F. Commercial Solutions

Current commercial tools fall into three categories:

Manual takeoff tools (PlanSwift [38], Bluebeam) require human operators to trace elements and apply counts. While accurate, they scale poorly and depend entirely on operator expertise.

Rule-based estimation systems (STACK [39]) embed framing rules in software code. Rule updates require vendor modifications, and the reasoning behind outputs is not auditable.

AI-powered takeoff tools (Togal.AI [40]) use computer vision to detect areas and count symbols, but typically output raw quantities without assembly reasoning, cut optimization, or compliance verification.

G. Research Questions

Based on the gaps identified above, this work addresses the following research questions:

1) **RQ1:** Can a domain-specific knowledge graph effectively capture the relationships between architectural plans, building codes, material specifications, and construction practices?
2) **RQ2:** How can specialized LLM agents collaborate to produce accurate, code-compliant material takeoffs while maintaining explainability?

3) **RQ3:** Can full provenance tracking from plan facts to BOM outputs improve trust and enable continuous improvement?
4) **RQ4:** How can cut-stock optimization be integrated with knowledge-based reasoning to minimize waste while respecting construction constraints?

H. Key Contributions

This proposal presents the following novel contributions:

1) **KG-Centered Architecture**: A knowledge graph backbone that externalizes construction rules, material catalogs, code requirements, and historical precedents—enabling inspection, versioning, and auditing of all system knowledge [11].
2) **Provenance-First Design**: Every BOM line item links to (a) plan facts, (b) applied inference rules, and (c) code references, supporting explainability and regulatory compliance.
3) **Multi-Agent Reasoning**: Specialized agents for extraction QA, component inference, compliance checking, and instruction generation [15], [18], with clear separation from deterministic optimization tools.
4) **Integrated Cut Optimization**: Stock selection and cutting as a first-class concern, using ILP solvers [26] with kerf handling and multi-stock support [23], with full traceability to source requirements.
5) **Continuous Improvement Loop**: Human corrections and project outcomes feed back into the KG as new precedents, enabling system-wide learning without retraining.

I. Differentiation Summary

Table I compares our approach to existing solutions across key capabilities.

TABLE I
COMPARISON WITH EXISTING APPROACHES

Capability	Existing	Ours
Knowledge storage	Code-embedded	KG-externalized
Rule versioning	Manual updates	Temporal validity
Provenance	None/limited	Full chain
Cut optimization	Separate/none	Integrated
Compliance	Manual check	Auto w/ citations
Learning	Static	Continuous

III. SYSTEM ARCHITECTURE OVERVIEW

Figure 1 summarizes the proposed architecture. The design follows three architectural patterns common in modern AI systems [41]:

(1) Retrieval-Augmented Generation (RAG)—agents retrieve relevant facts from the KG before generating outputs, grounding decisions in structured knowledge rather than parametric memory alone.

(2) Tool-Augmented Agents—LLM agents invoke deterministic tools (optimizer, exporter) for tasks requiring provable correctness [17].

(3) Blackboard Architecture—the KG serves as a shared workspace where specialized agents read inputs, write inferences, and coordinate asynchronously.

The core design principle is that the system is *KG-centered*: extracted plan facts and inferred construction intent are continuously grounded against a structured graph of rules, components, and historical builds.

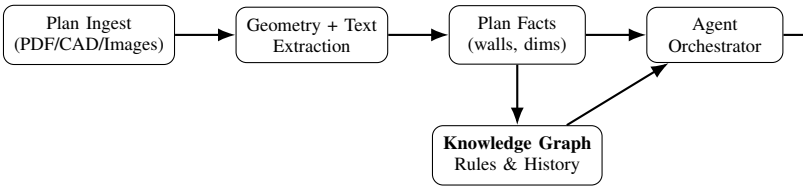

Fig. 1. KG-centered architecture: plan facts are extracted, normalized, stored with provenance, and used by agents to generate compliant, optimized outputs.

A. Key Modules

(1) Plan Ingest + Extraction. Convert drawings into machine-usable primitives: vector geometry (lines, polylines), detected room/wall boundaries, and text tokens (dimensions, callouts, schedules) [32]. The current implementation supports PDF (via PyMuPDF [36]), DXF (via ezdxf [42]), and DWG (via LibreDWG conversion). For raster images, YOLOv8 object detection [43] and EasyOCR [37] provide robust extraction.

(2) Plan Facts Model. Normalize extraction into a consistent intermediate representation: *WallSegments*, *Openings*, *Headers*, *Notes*, *Schedules*, and *Constraints* with explicit provenance back to the sheet, region, and confidence score.

(3) Knowledge Graph. Store rules, materials, codes, and historical knowledge (Section IV).

(4) Agent Orchestrator. A multi-agent workflow that queries the KG, performs reasoning and validation, and calls deterministic optimization routines for cutting and procurement.

(5) Export + Integration. Produce both human-usable artifacts (PDF/HTML summaries, build steps) and machine-readable exports (CSV/JSON/EDI-style mappings) suitable for lumber yard systems.

B. Data Flow

The system operates in five sequential phases:

Phase 1: Extraction. Plan documents are parsed into vector geometry, raster images, and text. YOLOv8 detects symbols; PyMuPDF extracts text and vectors from PDFs; ezdxf handles CAD formats. Output: raw geometry primitives and text tokens with bounding boxes.

Phase 2: Normalization. Raw extraction is normalized into `PlanFact` nodes in the KG, each with source provenance (sheet, region, confidence). Cross-validation checks dimension consistency and flags ambiguities.

Phase 3: Inference. Agents query the KG for plan facts and rules, then write inferred assemblies (`AssemblyIntent`,

`Component`) back to the KG. Each inference links to triggering facts and applied rules.

Phase 4: Optimization. The Procurement Agent selects stock items and invokes the cut-stock optimizer (ILP solver) for waste minimization. Results are written as `CutPiece` nodes linked to source components.

Phase 5: Export. Final outputs (BOM, cut lists, instructions) are generated by querying the KG and formatting results. Every output line item carries provenance links back through phases 4→3→2→1.

IV. Knowledge Graph Design

The knowledge graph (KG) serves as the backbone for *grounded, repeatable decisions* [6], [7]. Unlike rule-based systems that embed logic in code, the KG externalizes knowledge so it can be inspected, updated, and audited independently of the reasoning engine.

A. Theoretical Foundation

Knowledge graphs represent information as a labeled, directed multi-graph $G = (V, E, L)$ where vertices V represent entities, edges $E \subseteq V \times V$ represent relationships, and L provides type labels [6]. This formalism supports:

- **Explicit semantics**: Relationships are first-class citizens with types and properties
- **Schema flexibility**: New entity and relationship types can be added without migration
- **Path queries**: Multi-hop reasoning (e.g., "which code rules constrain this header?")
- **Provenance tracking**: Every fact can carry source attribution

Pan et al. [11] identify three paradigms for KG-LLM integration: KG-enhanced LLMs, LLM-augmented KGs, and synergized systems. Our architecture follows the synergized approach, where agents query the KG for grounded facts and update it with validated inferences.

B. Why Not Alternatives?

We considered three alternative knowledge representations:

Rule engines (e.g., Drools, CLIPS) excel at forward/backward chaining but embed rules in code or DSL files. Rules are difficult to inspect, version, and explain to end users. When rules conflict, debugging requires developer intervention.

Ontologies (e.g., OWL/RDF) provide formal semantics and reasoning via description logics [8]. However, OWL reasoners struggle with scale, closed-world assumptions conflict with construction's open-world nature, and the formalism is inaccessible to domain experts.

Relational databases handle structured data well but require JOIN-heavy queries for relationship traversal. A query like "find all components constrained by IRC R602.3" requires multiple joins across normalized tables; in a graph, it is a single-hop traversal.

The KG approach balances expressiveness, scalability, and maintainability for construction domain knowledge.

C. Knowledge Categories

The KG organizes construction knowledge into six categories:

- **Component definitions**: studs, headers, plates, cripples, blocking, sheathing—with parameterized assemblies
- **Material catalog**: dimensions, grades, treatments, SKUs, stock lengths, pricing, lead times
- **Fasteners and connectors**: nails, screws, straps, hangers, anchors—with application rules and schedules
- **Building rules**: framing conventions, span tables, loading assumptions, nailing patterns
- **Codes and regulations**: jurisdiction, version, section citations, applicability conditions
- **Historical precedents**: past project decisions, substitutions, waste rates, outcome feedback

D. Core Entities and Relationships

Table II summarizes the core entity types, and Table III shows key relationships.

TABLE II
CORE KG ENTITY TYPES

Entity Type	Description
Project	Building project with metadata
PlanSheet	Individual plan page with scale
PlanFact	Extracted fact with confidence
AssemblyIntent	Inferred assembly (e.g., header)
Component	Physical component instance
StockItem	Purchasable material
CutPiece	Result of cutting operation
CodeRule	Building code requirement
FastenerRule	Fastening specification
Supplier	Material supplier

TABLE III
CORE KG RELATIONSHIP TYPES

Relationship	From	To
hasSheet	Project	PlanSheet
mentions	PlanSheet	PlanFact
instantiates	PlanFact	AssemblyIntent
composedOf	AssemblyIntent	Component
builtFrom	Component	StockItem
cutInto	StockItem	CutPiece
constrainedBy	Component	CodeRule
requiresFastener	Component	FastenerRule
suppliedBy	StockItem	Supplier

E. Example Query: Provenance Chain

The following Cypher query retrieves the full provenance chain for a BOM line item, demonstrating the KG's traceability:

```
MATCH (bom:BOMItem {id: $itemId})
      -[:derivedFrom]->(cut:CutPiece)
      -[:cutFrom]->(stock:StockItem)
      <-[:builtFrom]-(comp:Component)
      <-[:composedOf]-(asm:AssemblyIntent)
      <-[:instantiates]-(fact:PlanFact)
      -[:extractedFrom]->(sheet:PlanSheet)
```

```
OPTIONAL MATCH (comp)-[:constrainedBy]->(code:CodeRule)
RETURN bom, cut, stock, comp, asm, fact,
       sheet, collect(code) as codeRules
```

This single query traverses the entire chain from BOM output back to plan source, collecting all applicable code rules—a query that would require 6+ JOINs in a relational model.

F. Versioning and Provenance

Construction knowledge evolves: codes are updated, suppliers change SKUs, practices differ by region. The KG supports this through:

- **Temporal validity**: Effective dates and regions on CodeRule and MaterialSpec nodes
- **Source provenance**: Document reference, section number, URL/file hash on every fact
- **Decision audit**: Agent decisions are logged with reasoning traces linked to KG facts

Every generated BOM line item links back to: (1) which plan facts triggered it, (2) which inference rules were applied, and (3) which code references constrained it.

G. Technology Choice: Neo4j

We select Neo4j as the graph database platform based on:

- **Native graph storage**: Index-free adjacency enables O(1) relationship traversal regardless of graph size [44]
- **Cypher query language**: Declarative pattern matching for complex multi-hop queries
- **Vector index support**: Native vector similarity search enables GraphRAG patterns [12], [41]
- **APOC procedures**: Graph algorithms (centrality, community detection) for analytics
- **Python ecosystem**: Async driver compatible with FastAPI; LangChain integration available

Alternative graph databases (Amazon Neptune, TigerGraph) were considered but Neo4j's developer tooling, community support, and vector search capabilities made it the pragmatic choice for this domain.

V. Data: Sources, Curation, and Ground Truth

A. Input Types

- **Vector CAD**: DXF/DWG files with layer-organized geometry
- **PDF Plans**: Both vector PDFs and scanned raster images
- **Schedules**: Door/window schedules, material legends
- **Specifications**: Project specs with material requirements
- **As-built data**: Jobsite photos, marked-up drawings

B. Ground Truth Strategy

1) Start with completed projects where final lumber orders and framing decisions are known
2) Align plan-derived assemblies with actual purchase orders and observed waste
3) Create manually labeled subset (50–100 residential sheets) verified by estimators
4) Capture human overrides as new precedents in the KG

C. Curation Loop

When the system encounters ambiguous cases (e.g., missing header callout, unclear wall type):

1) Propose options with confidence scores and citations
2) Record human selections back into the KG as new precedent
3) Track which rules produced correct vs. incorrect inferences
4) Continuously improve confidence calibration

VI. Agentic Workflow

Agents are responsible for *turning facts into decisions*, while deterministic tools handle optimization and exporting [13], [14], [15]. This separation ensures that reasoning is auditable and optimization is provably correct.

A. Design Principles

Our agent architecture follows three key principles from recent surveys on tool-augmented LLMs [18], [17]:

Reasoning-Action Separation. Following the ReAct pattern [16], each agent interleaves reasoning traces with tool invocations. Reasoning steps query the KG for facts and rules; actions invoke deterministic tools (optimizer, exporter) or update the KG with validated inferences. This enables auditability (decisions trace to KG facts), correctness (optimization uses proven algorithms), and debuggability (failed inferences can be replayed).

Specialization over Generalization. Rather than a single general-purpose agent, we deploy five specialized agents, each with domain-specific prompts, KG query templates, and tool access. Specialization improves accuracy by constraining the reasoning space.

Shared Memory via KG. Agents communicate through the knowledge graph rather than direct message passing. Each agent reads from and writes to the KG, creating a persistent, queryable record of all inferences.

B. Agent Architecture

Figure 2 shows the five agents and their workflow. Agents (blue) perform reasoning; tools (green) perform deterministic computation.

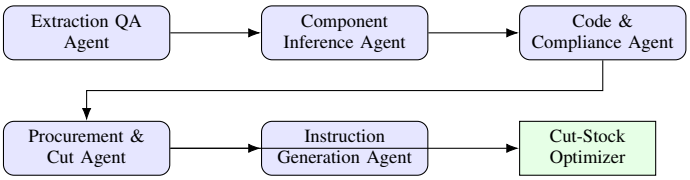

Fig. 2. Agent workflow: specialized agents (blue) handle reasoning; deterministic tools (green) handle optimization. All agents read/write to the shared KG.

C. Inter-Agent Communication

Agents communicate via the KG using a publish-subscribe pattern: (1) upstream agent writes inference results as new KG nodes; (2) downstream agent queries for nodes matching its input criteria; (3) status tracking via `pending`/`validated`/`flagged` properties; (4) provenance links to creating agent and reasoning trace. This decouples agents temporally and logically, enabling parallel execution and full audit trails.

D. Error Handling

Agents handle uncertainty through tiered strategies: **Confidence thresholds**—low-confidence results are flagged for human review rather than propagated. **Validation gates**—outputs are checked against schema constraints before KG writes. **Graceful degradation**—failed agents mark assemblies as "requires review" and continue processing unaffected portions.

E. Extraction QA Agent

Validates extracted geometry/text, flags low-confidence dimensions:

- Dimension consistency checks (wall lengths match scale)
- Missing element detection (walls without openings)
- OCR confidence thresholds (request higher DPI if <0.85)

F. Component Inference Agent

Maps validated plan facts to assemblies using KG rules:

- Wall type $\rightarrow$ stud size/spacing, plates, sheathing
- Opening size $\rightarrow$ header assembly, jack/king studs
- Bearing vs. non-bearing $\rightarrow$ header sizing requirements

G. Code & Compliance Agent

Checks assemblies against KG code rules [29], [31]:

- Header sizing for span/load per IRC [4]
- Stud spacing for load-bearing walls
- Fastener patterns; produces "compliant" or "requires review" with citations

H. Procurement & Cut Agent

Selects stock and invokes the cut optimizer:

- Stock lengths (8/10/12/14/16/20 ft, LVL, panels)
- Kerf allowance (0.125") and waste factors
- Substitution rules (prefer in-stock; longer stock if less waste)

I. Instruction Generation Agent

Produces build instructions with full traceability:

- Organized by wall line, room, or phase
- Component references with KG node IDs
- Fastener specs and inspection callouts with code citations

TABLE IV
PARSING TECHNOLOGIES

Format	Technology
PDF (vector)	PyMuPDF (fitz) [36]
PDF (raster)	pdf2image + OpenCV
DXF	ezdxf [42]
DWG	LibreDWG $\rightarrow$ DXF
OCR	EasyOCR [37], Tesseract
Object Detection	YOLOv8 [43]
Scale Detection	LLM vision (Claude/GPT-4)

VII. KEY TECHNOLOGIES

A. Parsing Stack

Plan parsing combines multiple technologies for robust multi-format support [32]:

Technology selection prioritizes open-source tools (PyMuPDF, ezdxf) where feasible, with commercial LLM APIs for tasks requiring reasoning.

B. Cut Optimization

The cutting stock problem is NP-hard but well-studied [23], [24]. We employ:

- **Heuristics**: First-Fit Decreasing (FFD) for fast, decent solutions
- **Exact methods**: Integer Linear Programming via OR-Tools [26] or PuLP
- **Kerf handling**: Each cut consumes blade width (0.125"); n pieces require $(n-1) \times$ kerf [25]
- **Multi-stock**: Optimize across available stock lengths simultaneously

Objective function:

$$\min \sum_{s \in S} c_s \cdot x_s + \lambda \cdot \text{waste} \tag{1}$$

where x_s is the count of stock length s, c_s is cost, and λ weights waste penalty.

C. Knowledge Graph Stack

Recent advances in LLM-KG integration [22] inform our technology choices:

- **Graph Database**: Neo4j Community/Enterprise
- **Query Language**: Cypher
- **Python Driver**: neo4j (async compatible)
- **Embeddings**: OpenAI/Anthropic for semantic search
- **Vector Index**: Neo4j vector index with GraphRAG patterns [12]

D. Agent Framework

- **LLM**: Claude [19] or GPT-4 [20]
- **Orchestration**: LangChain [21] or custom agent loop following ReAct patterns [16]
- **Tool Calling**: Structured function calls to KG, optimizer, exporters [17]
- **Memory**: Conversation context + KG query results

VIII. Outputs and Supplier Integration

A. Output Types

1) **Bill of Materials (BOM)**: Multi-level hierarchy (project → wall → assembly → stock items)
2) **Cut Lists**: Grouped by stock length and material type, with labeled cut pieces
3) **Fastener List**: Nails, screws, anchors, straps, hangers with quantities
4) **Build Instructions**: Step-by-step with plan sheet references and code notes
5) **Compliance Report**: Code citations, validation status, items requiring review

B. Export Formats

- **CSV**: Lumber yard import (SKU, qty, length, treatment, notes)
- **JSON**: Structured assemblies + cuts + provenance
- **PDF/HTML**: Human-readable reports with diagrams
- **EDI**: Optional mapping layer for specific suppliers

C. Sample JSON Structure

The following example shows a BOM entry with full provenance tracking:

```
{
  "project_id": "9557-Barnes-Road",
  "bom_items": [{
    "item_id": "BOM-142",
    "description": "2x10x12 DF-L #2",
    "qty": 2,
    "unit_cost": 18.75,
    "supplier_sku": "L2X10-12-DFL2",
    "cuts": [{
      "cut_id": "C-142-1",
      "length_in": 42,
      "label": "Front Door Header (1 of 2)",
      "kerf_in": 0.125
    }],
    "assembly": {
      "type": "header_assembly",
      "opening_id": "OP-003",
      "opening_label": "Front Door 3068",
      "span_in": 40,
      "load": "bearing",
      "components": ["(2) 2x10", "1/2\" ply spacer"],
      "code_rule": "IRC R602.7.2"
    },
    "provenance": {
      "plan_fact_id": "PF-203",
      "sheet": "A-101 Floor Plan",
      "region": [120, 450, 180, 480],
      "confidence": 0.94,
      "inferred_by": "ComponentInferenceAgent",
      "validated_by": "CodeComplianceAgent",
      "timestamp": "2024-01-15T14:32:18Z"
    }
  }],
  "waste_summary": {
    "total_stock_lineal_ft": 1248,
    "total_cut_lineal_ft": 1186,
    "waste_pct": 4.97
  }
}
```

This structure enables suppliers to import orders directly, framers to locate components on plans, and project managers to audit decisions back to source facts.

IX. Phased Development Plan

Development follows an iterative, test-driven approach common in AI system engineering [21]. Each phase delivers a working system with measurable improvements over the baseline.

A. Phase 1: KG Foundation + Enhanced Takeoff (4–6 weeks)

- Define initial ontology in Neo4j (entities, relationships)
- Migrate existing lumber calculations to KG-backed rules
- Load baseline component definitions and framing rules
- Parse 2–3 plan sets with provenance tracking
- Generate BOM with rule citations

Success Criteria: BOM accuracy >90% on validation set; all BOM items link to plan facts and rules; KG query latency <100ms.

B. Phase 2: Assembly Reasoning + Cut Optimization (6–10 weeks)

- Expand component library (headers, stud packs, plates, sheathing)
- Implement Component Inference Agent with KG queries
- Integrate OR-Tools cut-stock optimizer [26]
- Produce supplier-ready cut list exports
- Validate against ground truth projects

Success Criteria: Waste reduction to <8% (baseline: 15–20%); agent inference accuracy >85%; cut optimization runtime <30s per project.

C. Phase 3: Code/Regulation Layer (6–10 weeks)

- Model IRC/IBC code rules with jurisdiction and version [4]
- Implement Code & Compliance Agent following NLP extraction methods [29]
- Add compliance citations to outputs
- Build code rule editor for jurisdiction customization

Success Criteria: Code compliance validation accuracy >90%; all non-compliant assemblies flagged with specific IRC citations; support for 3+ jurisdictions.

D. Phase 4: Feedback Loop + Learning (Ongoing)

- Capture user overrides and corrections
- Store outcomes (actual waste, substitutions made)
- Continuously improve confidence calibration
- Expand KG with new precedents using continuous learning approaches [22]

Success Criteria: System accuracy improves >2% per 100 projects processed; user override rate decreases over time; KG grows with validated domain knowledge.

E. Risk Mitigation

Data quality: Start with high-quality architectural plans; flag low-confidence extractions for human review. **Code complexity:** Begin with core IRC framing rules; expand incrementally to specialty cases. **Integration:** Design JSON/CSV exports to match common supplier formats; iterate based on partner feedback.

X. Current Implementation Status

Building on established approaches for BIM-based quantity extraction [45], [46], the following components are already implemented in the construction-ai codebase:

TABLE V
IMPLEMENTATION STATUS

Component	Status	Phase
DXF/DWG Parsing	Complete	1
PDF Parsing	Complete	1
Wall Extraction	Complete	1
Stud/Plate Calculation	Complete	1
YOLOv8 Object Detection	Partial	2
Gemini Scale Detection	Complete	1
Web Interface (React)	Complete	1
PostgreSQL Database	Complete	1
Docker Infrastructure	Complete	1
Neo4j Knowledge Graph	**New**	1–2
Agent Orchestration	**New**	2
Cut Optimization	**New**	2
Code Compliance	**New**	3
Build Instructions	**New**	3

A. Implementation Gaps and Next Steps

The existing codebase provides a strong foundation for plan parsing and basic takeoff, but lacks: (1) externalized knowledge representation via KG, (2) multi-agent reasoning with provenance, (3) integrated cut optimization, and (4) code compliance verification.

The proposed phased approach (Section 8) builds on the working parser and web interface while adding the KG backbone and agent architecture that enable the key contributions outlined in Section 2a.

Immediate next steps focus on Phase 1: designing the initial KG schema, migrating existing lumber rules to graph representation, and establishing provenance tracking for all inferences.

XI. Scalability and Future Enhancements

A. Performance Targets

- Multi-plan uploads with concurrent processing
- Large residential/commercial plan sets (100+ sheets)
- Sub-minute takeoff for typical residential plans

B. Architecture for Scale

- Neo4j cluster for KG (read replicas for query load)
- Celery + Redis for async task processing
- Object storage (S3) for plan files and artifacts
- Kubernetes for horizontal scaling

C. Future Enhancements

- **Multi-trade expansion**: Drywall, flooring, roofing, MEP with trade-specific ontologies [8]
- **Crew-centric packaging**: Bundles per wall/area, labeled kits
- **Supplier pricing**: Order suggestions under cost/availability constraints
- **Automated RFI**: Generate requests for information when plans are inconsistent [30]
- **3D visualization**: Three.js framing model from takeoff data

XII. Business Case

A. Value Proposition

TABLE VI
EXPECTED OUTCOMES

Metric	Target
Takeoff time reduction	70–80%
Material waste reduction	15–20% $\rightarrow$ <5%
Extraction accuracy	95%+
Bid throughput increase	3–5$\times$
Change order reduction	90%

B. ROI Analysis

For a mid-size contractor processing 50 projects/year:

- **Time savings**: 40 hours/project $\times$ \$75/hr = \$3,000/project
- **Waste savings**: \$500–\$2,000/project depending on size
- **Annual benefit**: \$175,000–\$250,000
- **Payback period**: <6 months

XIII. Conclusion

A KG-centered design makes construction takeoff and cut optimization *auditable, improvable, and reusable*. By combining structured rules and historical precedents with agentic reasoning and deterministic optimization, Construction.AI can generate practical BOMs, waste-minimizing cut lists, and build instructions that connect directly to supplier workflows.

The system builds on a solid foundation of existing parsing and extraction capabilities, adding the knowledge graph layer that enables grounded reasoning and continuous improvement from real-world outcomes.

References

[1] McKinsey Global Institute, "Reinventing construction: A route to higher productivity," McKinsey & Company, Tech. Rep., 2017.

[2] Royal Institution of Chartered Surveyors, "Measurement and takeoff: Guidance note," RICS, Tech. Rep., 2019.

[3] Deloitte, "2023 engineering and construction industry outlook," Deloitte, Tech. Rep., 2023.

[4] International Code Council, *2021 International Residential Code (IRC)*. ICC, 2021.

[5] J. Manrique, G. Wadel, and R. Vázquez, "FRAMEX and CUTEX: A CAD-based system for automated wood framing and waste reduction," in *Proc. 24th Int. Symp. Automation and Robotics in Construction*, 2007, pp. 157–164.

[6] A. Hogan, E. Blomqvist, M. Cochez *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[7] Z. Zheng, Y. Zhou, X. Lu, and J. Lin, "Knowledge graph for the construction industry: A review," *Automation in Construction*, vol. 139, p. 104289, 2022.

[8] P. Pauwels and W. Terkaj, "Express to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology," in *Automation in Construction*, vol. 63, 2016, pp. 100–133.

[9] S. Wu, Q. Shen, Y. Deng, and J. Cheng, "Natural-language-based intelligent retrieval engine for BIM object database," *Computers in Industry*, vol. 132, p. 103528, 2021.

[10] W. Fang, L. Ma, P. E. D. Love, H. Luo, L. Ding, and A. Zhou, "Knowledge graph for identifying hazards on construction sites: Integrating computer vision with ontology," *Automation in Construction*, vol. 119, p. 103310, 2020.

[11] S. Pan, L. Luo, Y. Wang *et al.*, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[12] D. Edge, H. Trinh, N. Cheng *et al.*, "GraphRAG: From local to global retrieval-augmented generation," *arXiv preprint arXiv:2404.16130*, 2024.

[13] L. Wang, C. Ma, X. Feng *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[14] Z. Xi, W. Chen, X. Guo *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[15] J. Li, K. Zhang, H. Wang *et al.*, "Agentic large language models: A survey," *arXiv preprint arXiv:2503.23037*, 2025.

[16] S. Yao, J. Zhao, D. Yu *et al.*, "ReAct: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2023.

[17] T. Schick, J. Dwivedi-Yu, R. Dessì *et al.*, "Toolformer: Language models can teach themselves to use tools," in *NeurIPS*, 2023.

[18] L. Zhang, Y. Chen, X. Liu *et al.*, "LLM-based agents for tool learning: A survey," *Data Science and Engineering*, 2025.

[19] Anthropic, "Claude: Constitutional AI and tool use," 2024.

[20] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[21] H. Chase, "LangChain: Building applications with LLMs through composability," 2022.

[22] Y. Zhu, X. Wang, J. Chen *et al.*, "LLMs for knowledge graph construction and reasoning: Recent capabilities and future opportunities," *arXiv preprint arXiv:2305.13168*, 2024.

[23] P. C. Gilmore and R. E. Gomory, "A linear programming approach to the cutting-stock problem," *Operations Research*, vol. 9, no. 6, pp. 849–859, 1961.

[24] G. Wäscher, H. Haußner, and H. Schumann, "An improved typology of cutting and packing problems," *European Journal of Operational Research*, vol. 183, no. 3, pp. 1109–1130, 2007.

[25] Y. Cui, Y. P. Cui, and Y. Tang, "Sequential value correction heuristic for the one-dimensional cutting stock problem with multiple stock lengths," *Optimization Methods and Software*, vol. 32, no. 1, pp. 161–186, 2017.

[26] Google OR-Tools, "OR-Tools: Google's operations research tools," https://developers.google.com/optimization, 2024.

[27] W. Solihin and C. Eastman, "Classification of rules for automated BIM rule checking development," *Automation in Construction*, vol. 53, pp. 69–82, 2015.

[28] J. Dimyadi and R. Amor, "Automated building code compliance checking – where is it at?" *Proc. CIB W78*, 2013.

[29] J. Zhang and N. El-Gohary, "Semantic NLP-based information extraction from construction regulatory documents for automated compliance checking," *Journal of Computing in Civil Engineering*, vol. 30, no. 2, p. 04015014, 2016.

[30] J. Zhang and N. M. El-Gohary, "Automated information extraction from construction specifications for compliance checking," *Automation in Construction*, vol. 60, pp. 120–136, 2015.

[31] T. Beach, Y. Rezgui, H. Li, and T. Kasim, "A rule-based semantic approach for automated regulatory compliance in the construction sector," *Expert Systems with Applications*, vol. 42, no. 12, pp. 5219–5231, 2015.

[32] A. Pizarro *et al.*, "Survey on the automatic analysis of architectural floor plans," *Automation in Construction*, vol. 135, p. 104093, 2022.

[33] A. Kalervo, E. Prashnani *et al.*, "CubiCasa5K: A dataset and benchmark for floorplan image analysis," in *ICCV*, 2019.

[34] A. Rezvanifar *et al.*, "YOLO-based object detection of symbols in architectural floor plans," in *Int. Conf. Pattern Recognition (ICPR)*. IEEE, 2020, pp. 1234–1241.

[35] R. Jamieson, L. Chen, and A. Kumar, "Deep learning for automatic symbol detection in construction drawings," *Automation in Construction*, vol. 152, p. 104965, 2025.

[36] Artifex Software, "PyMuPDF: Python bindings for MuPDF," https://pymupdf.readthedocs.io/, 2024.

[37] JaidedAI, "EasyOCR: Ready-to-use OCR with 80+ languages," https://github.com/JaidedAI/EasyOCR, 2024.

[38] PlanSwift, "PlanSwift: Construction takeoff software," https://www.planswift.com, 2024.

[39] STACK Construction Technologies, "STACK: Cloud-based takeoff and estimating," https://www.stackct.com, 2024.

[40] Togal.AI, "Togal.AI: AI-powered construction takeoff," https://www.togal.ai, 2024.

[41] P. Lewis, E. Perez, A. Piktus *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *NeurIPS*, 2020.

[42] M. Tönjes, "ezdxf: A python package for DXF files," https://ezdxf.readthedocs.io/, 2024.

[43] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," 2023.

[44] Neo4j, Inc., "Graph databases in construction: Managing complex relationships," in *Neo4j Technical White Paper*, 2023.

[45] C. Eastman, P. Teicholz, R. Sacks, and K. Liston, *BIM Handbook: A Guide to Building Information Modeling*, 3rd ed. John Wiley & Sons, 2018.

[46] T. El-Diraby *et al.*, "Automated extraction of quantities from IFC building models," *J. Information Technology in Construction (ITcon)*, vol. 22, pp. 1–15, 2017.